# Inside OrionLang

A guided tour of the codebase

Interpreter · manual memory · execution trace · raylib visualizer

A 2–3 hour study guide for programmers new to this project

Written to match the current experimental implementation

# Contents


1. How to use this guide ..... 3
2. Repository map and the active path ..... 3
3. From characters to an AST ..... 4
   - 3.1. Tokens and scanning ..... 4
   - 3.2. The AST is a linked tree with named child slots ..... 4
   - 3.3. Parsing with precedence climbing ..... 5
4. How the interpreter executes programs ..... 6
   - 4.1. Runtime state and lexical scopes ..... 6
   - 4.2. Values, types, and expressions ..... 6
   - 4.3. Manual memory and the heap ..... 7
5. Capturing a replayable execution ..... 7
6. The raylib visualizer ..... 8
   - 6.1. Startup, input, and rendering loop ..... 8
   - 6.2. Flow chart: bottom-up sizing and orthogonal edges ..... 8
   - 6.3. Dynamic memory: placement, grouping, and routes ..... 9
7. The other VM and the listing: what they teach today ..... 9
8. Learning the data structures and algorithms through examples ..... 10
9. An exploration lab ..... 10
10. Extension guide: where a new feature travels ..... 11
11. Current limitations and design questions ..... 11
12. Quick reference ..... 11

# 1. How to use this guide

This document explains how the current project works by following a source file from bytes on disk through tokens, syntax tree, evaluation, memory, trace capture, and rendering. It emphasizes the code that runs today, while calling out older or unfinished subsystems that can look like the active compiler at first glance.

The goal is not to read every line in one sitting. Use the route below, keep the repository open, and run the examples as you go. The times include reading the relevant code and pausing to inspect a couple of executions.

| Part | Study task | Time |
|---|---|---|
| 1–2 | Repository map; language pipeline and parser | 25 min |
| 3–4 | Interpreter, scopes, calls, and expression evaluation | 35 min |
| 5 | Heap layout, allocator, and memory safety checks | 25 min |
| 6 | Trace format and snapshot lifecycle | 20 min |
| 7–8 | Visualizer, flow layout, dynamic memory placement and routing | 40 min |
| 9–10 | Examples, limits, and guided exploration | 20 min |
| Total | Read and explore at a steady pace | 165 min |

Useful code-reading habit: first identify a function's inputs and outputs; next mark who owns each pointer; then follow one concrete example through its callers. OrionLang is small enough that tracing one execution by hand teaches more than trying to understand every helper in isolation.

# 2. Repository map and the active path

The repository contains an experimental language, an AST interpreter, a small managed heap, an instruction-listing writer, a separate bytecode VM experiment, and a raylib visualizer. These pieces are related historically, but they do not all participate in one compiled pipeline.

| File or group | Role | What to know |
|---|---|---|
| `src/main.c` | Command-line driver for `ori_vm` | Reads one source file; calls lexer, parser, listing writer, then interpreter. |
| `src/lexer.c` | Scanner | Converts characters into `Token` records with source coordinates. |
| `src/language_parser.c` | Active parser | Builds the AST consumed by the current interpreter. |
| `src/interpreter.c` | Active language runtime | Evaluates expressions and statements, manages frames, and optionally captures trace snapshots. |
| `src/listing.c` | Readable instruction listing | Writes `build/<name>.bytecode.txt`. This is explanatory text, not executable bytecode. |
| `src/stack_visualizer.c` | Separate raylib app | Runs the traced interpreter first, then displays the captured execution. |
| `src/vm.c` and `include/orion/vm.h` | Bytecode VM experiment and heap API | The opcode dispatcher exists, but the current `ori_vm` path does not execute generated bytecode. The active AST interpreter does reuse the heap functions. |
| `src/codegen.c`, `src/parser.c` | Earlier experiments | Not included in the current Makefile's `ori_vm` source list. Read as historical prototypes, not as the language's present behavior. |

| | | |
|---|---|---|
| include/orion/ *.def.h | X-macro definition lists | Single lists of token, AST-node, operator, and bytecode-opcode definitions expanded by headers and implementation code. |
| examples/, tests/, docs/ | Learning material | Runnable Orion examples, C regression programs, and the language book. |

The current execution path is:

```
.ori source
    -> src/main.c or src/stack_visualizer.c
    -> lex_source                (src/lexer.c)
    -> parse_program             (src/language_parser.c)
    -> ASTNode tree
    -> run_program / run_program_traced (src/interpreter.c)
    -> heap helpers              (src/vm.c)
```

`ori_vm` asks `write_bytecode_listing` to produce an additional human-readable representation before it interprets the AST. The visualizer uses the same lexer and parser, calls `run_program_traced`, and only then opens a window. This design makes playback deterministic and simple to scrub, at the cost of running the whole program before the UI appears and retaining a snapshot for each traced statement.

The Makefile is part of the architecture: its source lists define which implementation is active. When in doubt about whether a C file is built, inspect `SRCS`, the visualizer target, and the test target instead of inferring from the filename.

# 3. From characters to an AST

## 3.1. Tokens and scanning

`include/orion/ast.h` defines both `Token` and `ASTNode`; `include/orion/tokens.def.h` is expanded into the `TokenType` enum. A token stores a kind, a pointer to its lexeme text, and a line and column. `TokenStream` in `include/orion/lexer.h` is a growable array of those records.

`lex_source` in `src/lexer.c` walks the source buffer from left to right. Its core decisions are conventional:

- Ignore whitespace and `//` comments while maintaining line and column counters.
- Prefer two-character forms such as `==`, `&&`, `->`, and `<=` before single-character tokens. This is the scanner's version of maximal munch: choose the longest valid operator at the current position.
- Scan numbers as integers or decimal/exponent floats, copy the consumed spelling into owned memory, and preserve that spelling as the token lexeme.
- Scan identifiers, then compare their spelling with reserved words such as `while`, `break`, and `alloc`.
- Append an explicit EOF token so parser lookahead always has a terminal item.

The scanner duplicates lexemes with `strdup`; `free_token_stream` releases each string and then the token array. The AST nodes keep token records whose lexeme pointers refer to those strings. That means the token stream must outlive parsing and execution; `main` currently frees it only after the program has run.

Character literals are deliberately simple: the scanner takes one byte between quotes. The implementation is not a full Unicode character or escape-sequence system. String tokens retain their quotes and are accepted by the runtime only as direct `print` arguments.

## 3.2. The AST is a linked tree with named child slots

`ASTNode` has five pointers: `left`, `right`, `extra`, and `next`, in addition to the node's token and type. `next` links siblings or sequential statements. The other child pointers have conventions that depend on node type. These conventions are the language's internal data model; when changing a parser branch, also check its interpreter, listing, trace, and visualizer consumers.

| Node | Child layout | Example interpretation |
|---|---|---|
| AST_PROGRAM | left: top-level statements; right: structs; extra: functions | The three top-level categories are held as sibling lists. |
| AST_FUNC_DECL | left: body; extra: parameter list; right: return-type metadata | Function definitions are stored in program->extra. |
| AST_STRUCT_DECL | left: fields | The type's ordered field declarations define its word layout. |
| AST_IF_STMT | left: condition; right: then body; extra: optional else body | Each body is another next-linked statement list. |
| AST_WHILE_STMT | left: condition; right: body | The interpreter reevaluates the condition before each iteration. |
| AST_VAR_DECL | left: identifier; right: initializer; extra: declared type metadata | The declaration token also carries the variable's spelling and source location. |
| AST_ASSIGN | left: destination expression; right: source expression | The destination may be a local, indexed element, or struct field. |
| AST_BINARY | left: operand; right: operand | The operator is opToken.type; logical operators are evaluated with short-circuit rules. |
| AST_UNARY | left: operand | The token identifies -, +, or !. |
| AST_CALL | left: argument list | The call's token contains its function name. |
| AST_INDEX_ACCESS | left: array expression; right: index | The runtime scales the index by one 64-bit word. |
| AST_FIELD_ACCESS | left: base expression | The field token names the member to load. |

include/orion/ast.def.h uses an X-macro list to generate the enum. Instead of manually maintaining several copies of the AST node names, code includes the same list with a different definition of macro X. This technique is useful when a fixed set of operations has several mechanically derived representations. It also means adding a node changes enum values: all translation units that include the definitions must be rebuilt together.

### 3.3. Parsing with precedence climbing

src/language_parser.c is the parser used by the current programs. It uses a Parser cursor over the token array. peek observes the current token, take consumes one, match consumes conditionally, and expect emits a source-positioned parse error if the next token is not the required kind.

Expressions use precedence climbing, a compact alternative to writing one parser function per precedence level. precedence maps each binary token to an integer. parse_expr(p, min_prec) parses a unary/primary expression, then repeatedly consumes operators whose precedence is at least min_prec. For a left-associative operator it recurses at prec + 1; this makes a chain like a - b - c group as (a - b) - c. Parentheses call back into parse_expr at the lowest threshold.

The implemented precedence order, from loose to tight, is ||, &&, equality, relational comparisons, shifts, addition/subtraction, then multiplication/division/remainder. Unary operators bind before all of those. Calls, indexing, and field selection are parsed as postfix forms attached to an identifier expression.

Statement parsing is direct recursive descent. parse_block consumes {, repeatedly calls parse_stmt, links returned nodes with next, and requires }. The parser tracks loop_depth while parsing a while body. This allows it to reject break and continue outside a loop and supports nesting: the interpreter then naturally sends a loop-control result to the nearest loop that consumes it.

Declarations are recognized from token shape. Primitive type tokens always start a declaration; a user-defined type is recognized in local declarations when an identifier is followed by * (for example Node* current). This heuristic is one reason the parser is intentionally small and the type system is not yet a complete static checker.

Parser errors call `exit(1)` rather than recovering to parse later statements. This makes the implementation easy to follow, but an editor integration would need a recovery strategy so one mistake does not end all analysis.

# 4. How the interpreter executes programs

## 4.1. Runtime state and lexical scopes

At the top of `src/interpreter.c`, `Runtime` groups the active heap VM, function and struct declaration lists, step counter, call-depth counter, optional trace destination, and heap-object metadata used by the visualizer. A `Frame` is a small fixed array of `Variable` entries plus a pointer to its parent frame. A variable records its source name, descriptive type, 64-bit value, synthetic display address, and whether it is a pointer.

There is no bytecode local-slot execution here. Local variables are ordinary C structs in `Frame.vars`. The parent link models nested lexical scopes: `find_var` searches the current frame and its parents, so an inner block can read or update an outer variable. A declaration is inserted in the current frame, and duplicate names in the same scope are rejected. The innermost matching name wins, implementing shadowing.

Every `if` branch gets a temporary `Frame scope`. Each `while` iteration creates a fresh scope for its body. Function calls create a frame whose parent is the caller's current frame, then bind each argument to the corresponding parameter. That parent link also means functions can currently see caller variables by name: this is dynamically convenient but differs from the usual rule that a function only sees its own lexical globals and parameters. It is an important design choice to revisit if OrionLang is meant to teach conventional lexical scope.

The frames themselves live on the C call stack and disappear when their C function returns. The interpreter copies the values needed by trace snapshots before that happens. This gives the visualizer persistent history without making OrionLang local variables actual C pointers.

## 4.2. Values, types, and expressions

All runtime values occupy a `uint64_t`. Integers are interpreted as signed 64-bit values at arithmetic and print sites. Floats use IEEE-754 double bits copied into and out of that word with `memcpy`; this avoids aliasing a `double*` over an integer object. A pointer is a numeric heap offset stored in the same word. The declared type is metadata used for display and a limited number of conversions, not a full static type system.

`expression_is_float` asks whether an expression should be interpreted as a float by inspecting literals, variable metadata, fields, array elements, call return metadata, and arithmetic subexpressions. `coerce_assignment` converts integer-to-float or truncates float-to-integer on assignment. Mixed arithmetic promotes the integer operand to `double`. Remainder and shifts reject float operands. Logical `&&` and `||` short-circuit: the right-hand expression is skipped when the left side determines the result.

`eval` is a recursive tree walk. Each case evaluates its children and computes or fetches a result:

- Literals turn their token text into integer, float-bit, character-code, string, or null values.
- Identifiers look up a variable and return its stored word.
- `AST_BINARY` evaluates operands, applies short-circuiting, then chooses integer or floating-point behavior.
- Calls either invoke the built-in `print` path or locate a user function, evaluate arguments, bind a frame, run its body, and return its result.
- Index and field expressions form a heap address and use the checked heap load helper.
- `alloc` allocates an array or struct and returns the payload address.

`assign` handles l-values separately from expression evaluation. It evaluates the right side, determines whether the destination is a local, array element, or field, applies available conversion rules, and writes the result. Keeping l-value resolution here is a small interpreter version of the distinction between an expression's value and the storage location an assignment modifies.

`execute_list` walks a `next`-linked statement sequence. It propagates a `Result` out of nested blocks. The result carries either a return value or a loop-control signal. A `while` consumes `break` and `continue`: break ends that loop;

continue starts the next condition check. Since an inner while consumes its own signal first, nested loops target the nearest loop without a separate loop stack in the runtime.

The runtime has explicit limits: local slots per frame, call depth, managed heap size, tracked trace objects, and a global execution-step limit. These bounds protect this educational interpreter from runaway recursion and loops, but are not a substitute for static analysis.

### 4.3. Manual memory and the heap

Orion pointer values are integer offsets into a fixed 1 MiB byte array owned by `VM`. Zero is null. A heap allocation returns the address of its payload, not its header. Arrays contain 8-byte elements; struct fields also occupy one 8-byte word apiece. Pointers are copied by value, so assigning one pointer to another creates an alias to the same object rather than cloning it.

`src/vm.c` implements an implicit free-list allocator. Each block is stored as:

```
heap byte array
  [HeapHeader][payload, padded to 8 bytes][HeapHeader][payload]...
                ^ returned address
```

The header stores the requested size, a free flag, a magic marker, and the block's physical capacity. Allocation rounds the payload capacity up to an 8-byte boundary. It scans existing blocks from the beginning and takes the first free block large enough (first fit). A reused block keeps its capacity, records the new requested size, is zero-filled, and is marked live. If no block fits, the allocator appends a new header and payload at `heap_top`. It does not split large blocks or coalesce adjacent free blocks. Thus reuse is simple but can fragment space.

`heap_free` walks the same block chain to ensure the address equals the start of a payload. It rejects addresses outside the heap, pointers into the middle of blocks, corrupt headers, and double frees. `heap_resolve` scans blocks to ensure a read or write range belongs wholly to one live payload; it detects out-of-bounds access and use-after-free. Loads and stores use `memcpy`, avoiding alignment assumptions about the byte array.

An array index is computed as `base + index * sizeof(uint64_t)`, with an overflow guard before the calculation. The heap knows the allocation's byte range, but the language's pointer itself has no array length. Access within the same allocated block is therefore possible even if it is beyond the original logical element count, provided it stays within the payload. The visualizer separately remembers the declared element count for display.

Struct allocation sizes the payload from the number of field declarations. Field access currently finds an offset by field name across all declared structs; if the same name occurs at different offsets, the runtime reports ambiguity. The visualizer has each object's actual struct declaration and can display field types, but the execution lookup is still name-centric. This is a clear candidate for a future type-aware field resolution design.

## 5. Capturing a replayable execution

`include/orion/trace.h` defines the boundary between interpreter and UI. It deliberately contains plain C data: strings in fixed-size buffers, numeric values and addresses, and owned arrays of frames, objects, and fields. Raylib types do not leak into the interpreter. This separation keeps `ori_vm` buildable without linking the graphics library.

When `run_program_traced` receives a trace, `trace_statement` appends a snapshot at the start of each executed statement. It records the source line and statement spelling, active call frames, visible variables, and all tracked heap objects and fields. It copies the values instead of retaining pointers into short-lived C stack frames. As expression evaluation proceeds, variable and field reads mark the corresponding entry in that current statement's snapshot. The UI compares adjacent snapshots to infer which values changed. Since snapshots contain pre-statement state, a write becomes visible in the following statement's snapshot; there is no separate final-state snapshot after the program ends.

This is statement-level replay, not a machine-instruction debugger. Internal operations such as evaluating each operand increment the runtime step budget but do not each create a new visible frame in the trace. A single source statement can therefore read several variables, allocate, and write memory while appearing as one UI step. The end state after the final statement is not necessarily represented as an extra separate snapshot.

The trace format is capped for bounded UI storage: at most 8 frames, 6 variables per frame, 16 tracked objects, and 64 fields per object are copied into a step. These are display limits and should be kept in mind when constructing very large examples. `orion_trace_free` releases every nested allocation and clears the trace object; this ownership function is essential because snapshots allocate per-step arrays.

Addresses shown for local variables are synthetic monotonically increasing display identifiers. They are not C stack addresses and are not dereferenceable by the Orion program. Heap addresses are actual offsets used by the custom allocator. Keeping these identities separate lets snapshots match a local variable across steps without exposing temporary interpreter addresses.

# 6. The raylib visualizer

## 6.1. Startup, input, and rendering loop

`src/stack_visualizer.c` is a separate executable. It reads one `.ori` file, splits the original text into lines, lexes and parses it, then executes once with tracing enabled. It initializes raylib only after execution has completed. The main window loop selects a trace index, derives the active source line and frame, handles inputs, lays out the selected views, and draws the result. N/Right/Space step forward; B/Left step back; R resets; P toggles timed playback; brackets adjust playback rate; F selects a function flow chart; M selects the dynamic memory view. On-screen controls mirror the basic transport actions.

Source text is clipped to a panel and scrolled. Font sizes are loaded at requested raster sizes and cached in a small least-recently-used cache; the intent is to avoid scaling a tiny raster glyph up or down across zoom levels. Raylib's texture filter is set to bilinear. Text fitting measures glyph extents and shortens long labels with an ellipsis. This is why the visualizer uses a supplied font asset and expects to be launched from the repository root.

The interface has two panels: source or flow chart on the left, and variable values or dynamic memory on the right. A source statement is highlighted from the current trace step. The variable/value view highlights changed values in warm gold and reads in blue. Keyboard repeat is implemented with elapsed-time timers rather than depending on the operating system's repeat settings. Playback uses an accumulator: frame time is added until one or more step periods have elapsed, then the trace index advances. This keeps playback speed roughly stable if frame rate varies.

## 6.2. Flow chart: bottom-up sizing and orthogonal edges

`FlowLayout` is a UI-side tree that pairs each statement AST node with geometry and child layouts. `flow_layout_build` recursively builds nested bodies first. Each node reports its own width and height plus the total width and height required by the entire subtree. For an `if`, the space estimate includes side-by-side then and else regions; for a `while`, it reserves room for the body and a return route. A sequential `next` statement is placed below the previous node's complete local region. This is a bottom-up layout: children establish the space their parent must reserve.

After measuring, `flow_layout_place` assigns positions. Branches get separate lanes, and loop bodies sit beside their condition so there is room for a visible back edge. Each subtree's bounding region is then computed recursively with clearance around it. The flow edge router uses these reserved corridors to draw 90-degree polygonal paths. It chooses explicit attachment points at the top, bottom, or side of nodes; arrowheads are drawn at the final segment. Nested region bounds act as an approximate obstacle hierarchy, keeping edges out of node boxes and reducing crossings without a general graph-routing library.

Nodes encode statement category with shape and color: conditions are diamonds, loops rounded boxes, returns ellipses, allocations/declarations and frees distinct polygons, and ordinary operations rounded rectangles. The active source node receives a stronger fill and text contrast. The view supports wheel/button zoom, panning, dragging individual nodes, and smooth follow when the current node moves outside the visible area. Layout offsets are keyed by AST-node identity so the user's manual placement survives redraws while that parsed program remains alive.

The flow view is a visualization of statement structure, not a compiler control-flow graph. Its source-line mapping and branch routes are approximate. In particular, adding a new control-transfer construct may require explicit routing logic if the diagram should show the exact edge for that construct rather than merely display its statement node.

### 6.3. Dynamic memory: placement, grouping, and routes

The memory view builds a temporary array of `MemoryNode` records for the current snapshot. Heap objects are considered first and receive stable allocation-order positions at the right of the panel. Stack variables are processed newest-first. A pointer variable starts near its target when that target is visible; collision relaxation then moves it away from existing boxes. `memory_relax_new_box` performs repeated minimum-overlap nudges with a small gap. If local nudges cannot guarantee separation, it moves the new item below the existing obstacles. This makes non-overlap a hard placement rule, with compactness as a secondary goal.

Variables belonging to a function are grouped into frame bounds. `memory_align_function_variables` aligns the group in one vertical column; `memory_pack_function_boxes` treats each function's bounds as an obstacle and separates the groups from heap boxes and sibling frames. Per-address and per-frame offsets preserve user drags while the trace changes. Popped stack variables remain as dim, slightly smaller ghost boxes to preserve recent context; current variables and frame boxes are drawn above the connectors.

Pointer edges connect local variables and struct/array fields to visible target addresses. When several nearby pointers share a target, `draw_memory_edges` groups their source ports and shares a route trunk. Routes use horizontal and vertical segments. A registry of already drawn segments detects perpendicular crossings and leaves a small visible gap on the later segment, so a crossing reads as two paths rather than a fused junction. Each route receives a circle at its origin and an arrowhead at its destination. The rendering order draws routes before boxes, so unrelated objects occlude the lines.

Arrays use an 8-column grid with one cell per displayed element. This allows an object to grow in both width and height; only the trace-format field cap limits how many values are available to this panel. Struct fields are nested inside their allocation box. Values retain their type, address, change/read state, and pointer target where known. For heap allocations, `memory_owner_history` scans earlier trace steps for variables that most recently started pointing at an object and keeps a short history of those names, subject to the box width. This is a display heuristic; the language does not implement ownership semantics.

The focus helper selects a changed or read variable/field, reconstructs the object's layout position using the same placement rules, and measures what fraction of that box is visible. If less than roughly 70 percent is visible (about 30 percent clipped), a damped interpolation moves the camera toward it. The exponential interpolation depends on frame time, so the motion eases smoothly instead of teleporting.

These layout functions are heuristics, not a force-directed graph solver. The design priorities are stable heap placement, no overlaps, frame grouping, pointer-target proximity, readable orthogonal edges, and only then short total distance. When improving layout, preserve determinism across adjacent trace steps: changing positions too much at every step makes the memory story harder to follow than a slightly longer arrow.

## 7. The other VM and the listing: what they teach today

`include/orion/opcode.def.h` lists opcodes for a register-based virtual machine: constants, arithmetic, comparisons, jumps, calls, allocation, field and element access, local slots, casts, printing, and traps. `src/vm.c` contains an interpreter loop that reads opcodes and updates registers and the instruction pointer. Its heap allocator is also reused by the active AST interpreter.

However, the active command-line pipeline does not call `vm_execute`, and the current listing writer does not encode a binary program for it. `src/listing.c` walks the AST and prints readable mnemonic-like lines, including labels for conditionals and loops. It is useful for learning what a future compiler might emit, but the project README correctly describes it as a text listing. Treat the bytecode engine and older `src/codegen.c` as work in progress until a real code-generation path, instruction encoding, and end-to-end tests connect them to Orion source.

The repository also has `src/parser.c`, a separate early parser with overlapping helper names and an older AST interpretation. It is not the parser linked into `ori_vm`; the active implementation is `src/language_parser.c`. Knowing this avoids a common exploration trap: a file can look like the compiler's heart but be absent from the build target.

## 8. Learning the data structures and algorithms through examples

The examples are small end-to-end tests of language concepts, each with an algorithmic invariant worth noticing.

| Example | Core idea | Time | Extra space |
|---|---|---|---|
| `two_sum.ori` | Enumerate pairs with nested loops; release result and input arrays. | $O(n^2)$ | O(1) beyond input/ output |
| `binary_search.ori` | Maintain a sorted inclusive search interval and discard half each round. | O(log n) | O(1) |
| `reverse_list.ori` | Rewire each `next` pointer using previous/current/following invariants. | O(n) | O(1) |
| `merge_sort.ori` | Divide into halves; merge sorted runs through a reusable scratch array. | O(n log n) | O(n) |
| `towers_of_hanoi.ori` | Move n disks by solving two n−1 subproblems around one move. | O(2^n) | O(n) call depth |
| `example_code.ori` | A broad tour: arrays, functions, recursion, floats/integers, structs, pointers, loops, break/continue, and frees. | Mixed | Depends on feature |

For merge sort, the recurrence is `T(n) = 2T(n/2) + θ(n)`, which gives `θ(n log n)`. The scratch allocation is reused across recursive calls, so the auxiliary array space remains linear rather than allocating a fresh array at each level. For Hanoi, `T(n) = 2T(n−1) + 1`, giving `2^n − 1` moves and linear recursion depth. The visualizer makes the recursion frames and pointer-backed scratch array concrete.

The language is designed for algorithm practice, but its current type and library surface is still small. It has no modules, general strings, input API, hash map, or `for` loop. `break` and `continue` are supported inside `while` loops. Static type checking remains limited. Many Easy problems can be expressed; others become awkward when they assume a standard library collection or richer strings. The examples show the supported style, not a promise of full LeetCode compatibility.

## 9. An exploration lab

Try these short investigations while reading the source. They are intended as code-reading exercises, not assignments to change the project.

1. In `examples/two_sum.ori`, follow `answer[0] = i` from parsing through the `AST_ASSIGN` node, `assign`, `heap_store_u64`, and the snapshot's object fields. Write down the exact address formula.
2. In `examples/reverse_list.ori`, draw the three-node list before and after one loop iteration. Identify why `following` must be saved before overwriting `current.next`.
3. In `examples/merge_sort.ori`, identify the half-open range convention `[left, right)`. Explain why a one-element or empty range satisfies the base case.
4. In `src/interpreter.c`, trace `if (i == 2) continue;` through the returned `Result`, the `AST_IF_STMT` propagation, and the nearest `AST_WHILE_STMT` consumption.
5. In `src/vm.c`, simulate an allocation, free, and same-size allocation. Which header fields change? Why does the payload address remain stable on reuse?
6. In `src/stack_visualizer.c`, find one box-placement rule and one edge-routing rule. Change a local constant in your head and predict what becomes less readable.
7. Compare `make ori_vm` with `make ori_stack_viz`. Which translation units are linked into each? Which functions only appear in the bytecode experiment?

## 10. Extension guide: where a new feature travels

Adding a language feature is a cross-layer task. A keyword such as `break` needs a token kind in `tokens.def.h`, scanner recognition in `lexer.c`, a node kind in `ast.def.h`, parser construction and context checks in `language_parser.c`, runtime semantics in `interpreter.c`, and a readable representation in `listing.c`. If the visualizer should explain it, review `flow_label`, AST traversal, and edge construction in `stack_visualizer.c`. Update README/spec/tutorial text after behavior settles.

For a new expression operator, decide its precedence and associativity first. Add scanner support, parser precedence, AST representation, interpreter semantics, listing representation, and an example. Short-circuit operators are a special case: evaluating both sides eagerly would change observable behavior, so they must branch before evaluating the right side.

For a new heap object shape, decide its in-memory layout and alignment before writing drawing code. Keep the runtime's access validation authoritative; do not let the visualizer infer that a pointer is safe simply because an address resembles a rendered box. Extend trace records with copied, UI-neutral data, then teach rendering and history code how to display that data.

For a new visual layout behavior, preserve a strict order: gather the snapshot; construct live nodes; place objects and frames; apply saved drag offsets; compute bounds; route arrows; draw boxes over arrows; then move focus smoothly if enough of the target is off-screen. Separating these phases makes layout bugs easier to localize than mixing hit-testing, geometry, route construction, and drawing into one loop.

## 11. Current limitations and design questions

This section distinguishes present behavior from intended direction. Treat it as a map for future contributors, not as a criticism of an experimental project.

- The AST interpreter is authoritative; generated listings and the separate bytecode engine are not yet a full compiler pipeline.
- The runtime stores all values in one 64-bit word. Type annotations aid display and some float coercions, but cannot prevent every invalid operation.
- Function frames link to caller frames, so name lookup can see caller locals. Decide whether to keep this as a teaching convenience or move toward lexical/module scope.
- Struct-field lookup by global field name becomes ambiguous when same-named fields occur at different offsets. A struct-aware field resolution node would remove that ambiguity.
- Heap pointers are raw offsets and do not carry allocation lengths. Runtime checks catch accesses outside known blocks, but not every logical array-bound violation.
- Heap allocation uses first fit without splitting or coalescing. This keeps the allocator teachable and compact while leaving fragmentation behavior visible.
- Trace data is copied eagerly and bounded by fixed maxima. Larger programs can outgrow the visualized subset or consume substantial memory because each statement stores a broad memory snapshot.
- The visualizer solves a two-dimensional layout problem with hand-written constraints and orthogonal routes. It favors explainable, repeatable placement over globally optimal compactness.
- The older parser and code generator are not wired into the current Makefile. Consolidate or archive them once their intended future role is clear.

Good next steps for someone learning the project are to add focused parser/interpreter regressions for nested loop control; create a feature-to-file checklist; separate type metadata from runtime words; and design a typed struct-field access representation. For the visualizer, first make layouts testable as pure geometry data, then add small deterministic scenarios for overlap and crossing behavior.

## 12. Quick reference

Build and run the interpreter:

```
make
./ori_vm examples/hello.ori
./ori_vm examples/merge_sort.ori
```

Build and open the stepper:

```
make ori_stack_viz
./ori_stack_viz examples/reverse_list.ori
```

Typical controls are `N` / Right / Space for next, `B` / Left for back, `R` for reset, `P` for play/pause, brackets for playback rate, `F` for flow view, `M` for dynamic memory, and `+` / `-` for zoom. Drag in the flow and memory panels to reposition nodes or boxes. Run from the repository root so relative asset paths resolve.

The most productive first reading sequence in the source is:

```
src/main.c
  -> src/lexer.c: lex_source
  -> src/language_parser.c: parse_program / parse_stmt / parse_expr
  -> include/orion/ast.def.h and include/orion/ast.h
  -> src/interpreter.c: run_program_internal / execute_list / eval / assign
  -> src/vm.c: heap_alloc / heap_free / heap_resolve
  -> include/orion/trace.h and src/interpreter.c: trace_statement
  -> src/stack_visualizer.c: main / flow_layout_build / draw_memory_view
```

If you can explain that route and the two different meanings of “address” in the visualizer, you have the core mental model needed to make useful changes to OrionLang.